1. Visão geral e marca

Marca: Moto Agora. Valores: dinamismo, rapidez e confiança. Identidade visual baseada nas letras "M" e "A" (abstratas e conectadas).

Sistema: Rastreamento e gestão de motos — cadastro de clientes e motos, controle de KM, manutenção, cobranças e integração WhatsApp.

2. Stack tecnológica

- **Frontend:** React, TypeScript, Vite, Axios, React Query
- **Backend:** Python, FastAPI, SQLAlchemy, Alembic, Pydantic
- **Infra:** PostgreSQL, Redis, Docker
- **Integrações:** API WhatsApp, SGLock (financeiro)

3. Estrutura geral do projeto (monorepo)

```
projeto_motos/
  backend/
  frontend/
  docker/
  docs/
```

4. Backend Python — estrutura de pastas

```
backend/
  app/
    main.py
    core/          → config.py, security.py, database.py, redis_client.py
    models/        → cliente_model.py, moto_model.py, manutencao_model.py, cobranca_model.p
    schemas/       → cliente_schema.py, moto_schema.py, manutencao_schema.py, cobranca_sche
    routes/        → cliente_routes, moto_routes, manutencao_routes, cobranca_routes, dashb
    services/      → whatsapp_service, manutencao_service, rastreamento_service, cobranca_s
    jobs/          → verificar_troca_oleo.py, verificar_cobrancas.py
    repositories/  → cliente_repository.py, moto_repository.py
  requirements.txt
```

4.1 Ambiente virtual

Dentro de `backend/` : `python -m venv venv` → ativar `venv\Scripts\activate` → `pip install fastapi uvicorn sqlalchemy psycopg2-binary redis python-dotenv alembic pydantic`

5. Banco de dados — modelos

Cliente

Campos: id, nome, telefone, cpf, endereco.

Moto

Campos: id, placa, modelo, odometro_atual, km_proxima_troca_oleo, endereco_pernoite, cliente_id (FK).

Manutenção

Campos: id, moto_id, tipo, descricao, custo, km_realizada.

Cobrança

Campos: id, cliente_id, valor, data_vencimento, boleto_url, status. Status: PENDENTE | PAGO | ATRASADO.

6. Conexão e Redis

PostgreSQL: `postgresql://postgres:postgres@localhost:5432/motos` — engine + SessionLocal + Base (SQLAlchemy).

Redis: host localhost:6379 — uso: fila envio WhatsApp, cache dashboard, jobs agendados.

7. Jobs automáticos

- **Troca de óleo:** se `odometro_atual >= km_proxima_troca_oleo` → buscar motos → verificar KM → enviar alerta WhatsApp.
- **Cobranças:** verificar vencimentos e disparar lembretes (ex.: vence amanhã → WhatsApp).

8. Serviço WhatsApp

Função: `async def enviar_mensagem(numero, mensagem)`. Exemplo de mensagem de manutenção:

⚠️ Aviso de manutenção — Moto: ABC1234 — KM atual: 10.200 — Necessário trocar óleo.

9. Backend inicial (main.py)

FastAPI app com `include_router` para rotas (moto, cliente, etc.). Rodar: `uvicorn app.main:app --reload`.

10. Frontend React — estrutura

- **pages:** Dashboard, Motos, Manutencao, Financeiro
- **components:** MotoTable, MotoForm, DashboardCards
- **services:** api.ts (axios baseURL localhost:8000), motos_service, manutencao_service

Dashboard: total motos, manutenção pendente, cobranças atrasadas, local pernoite.

Motos: cadastrar, editar, atualizar KM, registrar pernoite — campos: placa, modelo, odometro, km troca óleo, endereço pernoite.

11. Docker

Pasta `docker/` com `docker-compose.yml`: serviço **postgres** (postgres:15, porta 5432, DB motos) e **redis** (redis:7, porta 6379). Comando: `docker compose up -d`.

12. Fluxos principais

Cadastro: Cliente → Moto → Endereço pernoite → Controle KM.

Manutenção: KM alcançado → alerta → registro manutenção.

Financeiro: cobrança criada → vence amanhã → WhatsApp enviado.

13. Recomendação SaaS (empresa_id)

Incluir `empresa_id` nas tabelas principais para multi-tenant: uma empresa, várias motos e clientes. Facilita evolução para SaaS.